

CLOUD SECURITY FOR BEGINNERS

UNDERSTAND. SECURE. BUILD. GROW.
A PRACTICAL GUIDE FOR ASPIRING CLOUD AND DEVOPS ENGINEERS



SECURE TODAY. BUILD TOMORROW.

WHAT IS CLOUD SECURITY?

UNDERSTAND. SECURE. BUILD. GROW.

1. WHAT IS CLOUD SECURITY?

- Cloud security means protecting cloud environments, data, applications, networks and infrastructure from unauthorized access, attacks, misuse and accidental loss.
- It includes people, processes, technologies and policies that keep your cloud resources safe, available and resilient.



2. WHAT ARE WE PROTECTING?

In the cloud, we protect several key components:



Data – files, databases, backups, customer information.



Identities – users, roles, service accounts, credentials and permissions.



Applications – web apps, APIs, containers, serverless functions.



Networks – VPCs, subnets, firewalls, load balancers, internet access.



Infrastructure – virtual machines, containers, storage, databases and managed cloud services.

3. CONFIDENTIALITY, INTEGRITY, AVAILABILITY

Cloud security aims to protect the CIA triad:



CONFIDENTIALITY

Only authorized people can access the data.



INTEGRITY

Data is accurate and not tampered with.



AVAILABILITY

Data and services are available when needed.

4. SECURITY VS COMPLIANCE

SECURITY

- Focuses on protecting your systems and data.
- It is about how to prevent, detect and respond to risks.
- It is technical and operational.

COMPLIANCE

- Means following laws, regulations and industry standards (e.g. GDPR, HIPAA, ISO).
- It proves you are meeting required rules.
- Compliance supports security, but it is not the same thing.

5. WHY IT MATTERS TO DEVOPS ENGINEERS

- You deploy and manage the infrastructure.
- You work with cloud tools, IAM, networking and storage.
- You build and run CI/CD pipelines.
- You help keep applications running in production.
- Security misconfigurations can cause major incidents (e.g. data breaches, service outages, financial loss).
- Understanding security helps you build, deploy and operate systems safely from day one.

6. SIMPLE MENTAL MODEL

Think of cloud security as five connected pillars:



Identity

Who can access?



Network

How can they connect?



Workload

What runs the application?



Data

What needs to be protected?



Visibility

Can you see and detect problems?

ALL FIVE WORK TOGETHER
TO KEEP YOUR CLOUD ENVIRONMENT SECURE

CLOUD SECURITY IS NOT A FEATURE. IT IS A MINDSET.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

HOW CLOUD SECURITY IS DIFFERENT

SAME SECURITY PRINCIPLES. A DIFFERENT ENVIRONMENT.

1. TRADITIONAL vs CLOUD INFRASTRUCTURE

TRADITIONAL INFRASTRUCTURE



- You buy and manage hardware
- Setup takes weeks or months
- Fixed capacity
- Mostly in your own data center
- Physical access is required
- Scaling requires new hardware

CLOUD INFRASTRUCTURE



- You use resources from a provider
- Setup takes minutes
- On-demand and elastic
- No physical hardware to manage
- Access over the internet
- Scale up or down anytime

2. PHYSICAL vs LOGICAL SECURITY

PHYSICAL SECURITY (Traditional)



- Focuses on physical access
- Locked facilities
- Security guards
- CCTV and alarms
- Hardware protection
- Environmental controls (power, cooling, fire)

LOGICAL SECURITY (Cloud)



- Focuses on digital access
- Identity and access control
- Network security
- Encryption
- Logging and monitoring
- Configuration management
- Policy and compliance

3. APIs AND AUTOMATION



- Cloud is managed through APIs and automation, not just a GUI.
- APIs make it easy to create, change or delete resources.
- Automation (IaC, scripts, CI/CD) can introduce security risks if not properly controlled.
- You must secure API access, credentials and automation tools.

4. INFRASTRUCTURE IN MINUTES



- In the cloud, you can create servers, storage, databases and networks in minutes.
- This speed is great for development and DevOps.
- It also means mistakes can be made quickly, including insecure configurations.
- What used to take months now takes one click.

5. DYNAMIC RESOURCES



- Cloud resources are often temporary and constantly changing.
- Instances can be created and deleted automatically.
- This makes traditional, static security controls harder to apply.
- You need continuous monitoring and automated security checks.

6. REMOTE ADMINISTRATIVE ACCESS



- Cloud resources are managed over the internet.
- Engineers, admins and services access resources remotely.
- Remote access increases the risk of unauthorized access if not properly secured.
- Use strong authentication, MFA and least privilege.

7. MISCONFIGURATION AS A MAJOR RISK



- In traditional environments, physical controls limited exposure.
- In the cloud, misconfigurations are one of the biggest security risks.
- Examples: public storage buckets, open security groups, excessive IAM permissions, exposed databases.
- Most cloud security incidents happen due to misconfiguration, not hacking.

8. WHY CLOUD SPEED CHANGES SECURITY



- The speed and flexibility of the cloud increase both opportunities and risks.
- Resources can be created, changed or deleted in seconds.
- Attackers also move quickly and automate their attacks.
- This means security must be built into every stage: design, deployment, monitoring and operations.

SAME CLOUD. HIGHER SPEED. HIGHER RESPONSIBILITY.

THE SHARED RESPONSIBILITY MODEL

SECURITY IS A SHARED JOB. KNOW WHO DOES WHAT.

1. WHAT IS THE SHARED RESPONSIBILITY MODEL?

- The shared responsibility model defines which security tasks are handled by the cloud provider and which are handled by you, the customer.
- Cloud providers secure the cloud infrastructure, while customers secure what they build and run in the cloud.



2. SECURITY "OF" THE CLOUD vs "IN" THE CLOUD

SECURITY "OF" THE CLOUD

- Security of the cloud means the cloud provider secures the underlying infrastructure.
- Includes data centers, physical hardware, network, power, cooling and global infrastructure.



CLOUD PROVIDER
RESPONSIBILITY

SECURITY "IN" THE CLOUD

- Security in the cloud means you secure what you put in the cloud.
- Includes your data, applications, configurations, access, identities and security settings.



CUSTOMER
RESPONSIBILITY

3. IAAS, PAAS AND SAAS DIFFERENCES

Service Model	Provider Manages	You Manage
IaaS (Infrastructure as a Service)	Physical servers, network, storage, virtualization	Operating system, applications, data, identities, security configuration
PaaS (Platform as a Service)	Infrastructure, OS, runtime, middleware	Applications, data, identities, configuration
SaaS (Software as a Service)	Everything (infrastructure, OS, application, etc.)	Data, user access, configuration, compliance

5. WHY MANAGED SERVICES CHANGE RESPONSIBILITY

- With managed services (e.g. databases, serverless, Kubernetes), the cloud provider manages more of the infrastructure and platform components.
- However, you are still responsible for:
 - Your data
 - Your access and permissions
 - Your application configuration
 - Your security settings (e.g. encryption, network access)
- Managed services reduce operational work, but they do not remove your security responsibilities.



4. CLOUD PROVIDER vs CUSTOMER RESPONSIBILITIES

CLOUD PROVIDER USUALLY SECURES

- Data centers and physical security
- Hardware and infrastructure
- Networking and global connectivity
- Virtualization layer
- Availability and resilience of the cloud platform



CLOUD PROVIDER

CUSTOMER USUALLY SECURES

- Your data and content
- Identities, users and access management
- Applications and workloads
- Network configuration (e.g. security groups, firewalls)
- Operating systems and patches (in IaaS)
- Security configurations
- Compliance and data governance



CUSTOMER

6. COMMON BEGINNER MISUNDERSTANDING



- Many beginners think "the cloud provider secures everything."
- This is incorrect. You are responsible for securing your data, applications, identities and configurations.
- The cloud provider cannot protect you from misconfigurations inside your account.

7. REAL-WORLD MISCONFIGURATION EXAMPLE



- A company accidentally made an object storage bucket public.
- Sensitive customer files were exposed to the internet.
- This was not a cloud provider failure.
- It was a customer misconfiguration.
- The cloud provider secured the storage service, but the customer did not correctly configure the access settings.

8. KEY TAKEAWAY



- Understand your responsibilities.
- Secure what you build in the cloud.
- Always review configurations.
- Use least privilege.
- Assume misconfigurations can and will happen.
- Security is a shared job.

SAME CLOUD. SHARED RESPONSIBILITY. STRONGER SECURITY.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

CLOUD IDENTITY FUNDAMENTALS

IDENTITY IS THE FOUNDATION OF CLOUD SECURITY.

KNOW WHO YOU ARE. CONTROL WHAT THEY CAN DO.

1. WHAT IS AN IDENTITY?

- An identity is a unique representation of a person, application or service that can access cloud resources.
- It tells the cloud platform who or what is making a request.
- Every action in the cloud is linked to an identity.



2. HUMAN vs WORKLOAD IDENTITIES

HUMAN IDENTITIES



- Real people (engineers, admins, users).
- Use usernames, passwords and MFA.
- Access the cloud console, CLI or tools.
- Example: your IAM user account.

WORKLOAD IDENTITIES



- Non-human identities used by applications, services and tools.
- Use roles, service accounts or managed identities.
- Access cloud resources automatically.
- Example: EC2 role, Kubernetes service account, CI/CD pipeline identity.

3. AUTHENTICATION vs AUTHORIZATION

AUTHENTICATION



- Proves who you are.
- Uses passwords, MFA, access keys or tokens.
- Answers the question: "Are you who you say you are?"

AUTHORIZATION



- Determines what you can do.
- Based on permissions, policies and roles.
- Answers the question: "What are you allowed to do?"

4. USERS, GROUPS, ROLES

USERS



- Individual identities for people.
- Have credentials to sign in.
- Example: jane.doe

GROUPS



- Collection of users.
- Makes permission management easier.
- Example: DevOps Team

ROLES



- Grant permissions to users or services.
- Can be temporary or long-term.
- Example: ReadOnly Role, Admin Role

5. POLICIES AND PERMISSIONS



- Policies define what actions are allowed or denied.
- Permissions control access to specific cloud services and resources.
- You can attach policies to users, groups, roles or service accounts.
- Use least privilege (only the minimum permissions needed).

6. SERVICE ACCOUNTS



- Service accounts are special identities for applications and services.
- Used by workloads like VMs, containers, CI/CD pipelines and serverless functions.
- They allow systems to access cloud resources without using human credentials.
- Keep them secure and use least privilege.

7. WHY IDENTITY IS A CLOUD SECURITY BOUNDARY



- Identity controls who can access your cloud environment.
- If an identity is compromised, attackers can access your resources.
- Most cloud security incidents involve stolen or misused identities.
- Proper identity management reduces the risk of data breaches, unauthorized access and accidental changes.

8. SIMPLE FLOW: HOW IDENTITY WORKS

- A user or service authenticates.
- The identity is verified.
- The system checks permissions.
- If allowed, the identity can access the cloud resource.
- All actions are logged for auditing.

IDENTITY FLOW



SECURE IDENTITIES. SECURE YOUR CLOUD. SECURE YOUR FUTURE.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

LEAST PRIVILEGE AND IAM SECURITY

GIVE ONLY THE ACCESS NEEDED. NOTHING MORE.

1. WHAT LEAST PRIVILEGE MEANS

- Least privilege means giving users, roles or services only the minimum permissions they need to do their job.
- It reduces the risk of accidental changes and security breaches.
- If it is not needed, do not give it.



2. WHY "ADMINISTRATOR" SHOULD NOT BE THE DEFAULT

- Administrator access gives full control over your cloud environment.
- If credentials are compromised, an attacker can do anything.
- Most day-to-day tasks do not require administrator access.
- Use administrator access only when necessary and for a short time.



DO NOT USE
ADMINISTRATOR
AS A DEFAULT

3. ROLE-BASED ACCESS

- Use roles to assign permissions based on job functions.
- Examples: DevOps role, read-only role, support role, billing role.
- Roles make it easier to manage access and follow the principle of least privilege.



RIGHT ROLE
RIGHT ACCESS

4. TEMPORARY PERMISSIONS

- Use temporary access for administrative tasks instead of permanent access.
- Temporary permissions reduce the risk of long-term credential abuse.
- Examples: time-limited roles, just-in-time (JIT) access, session credentials.



ACCESS
ONLY WHEN
NEEDED

5. PERMISSION SCOPE

- Define how far a permission applies.
- Scope can be the entire account, a specific region or a specific resource (e.g. one bucket or one server).
- Use the narrowest scope possible.



NARROW SCOPE
LOWER RISK

6. EXCESSIVE PERMISSIONS

- Excessive permissions give users more access than they need.
- This increases the blast radius if an account is compromised.
- Regularly review permissions to remove unnecessary access.



MORE PERMISSIONS
= HIGHER RISK

7. PRIVILEGE ESCALATION

- Privilege escalation happens when a user gains higher permissions than intended.
- It can happen through misconfigured policies, overly permissive roles or vulnerable services.
- Always follow least privilege and monitor for unusual activity.



PREVENT
ESCALATION

8. PERIODIC ACCESS REVIEWS

- Regularly review who has access, what they can do and whether they still need it.
- Remove access for users who have changed roles or left the organization.
- Use automated tools where possible.



REVIEW
REMOVE
STAY SECURE

9. REMOVING UNUSED ACCESS



- Disable or delete unused users, roles, access keys and service accounts.
- Unused access can be abused if credentials are leaked.
- Clean up old permissions, stale roles and inactive accounts.
- Make access reviews part of your normal operations.

REMOVE
WHAT YOU NO LONGER
NEED

LESS ACCESS. LESS RISK. A MORE SECURE CLOUD.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

PASSWORDS, MFA, KEYS, AND CREDENTIALS

SECURE CREDENTIALS. PROTECT YOUR CLOUD. STOP ATTACKERS.

1. PASSWORD AUTHENTICATION

- Passwords are used to verify your identity when logging in.
- Use strong, unique passwords.
- Avoid common passwords and never reuse passwords.
- Use a password manager.
- Combine with MFA for better security.



**STRONG
PASSWORDS
KEEP ATTACKERS OUT**

2. MULTI-FACTOR AUTHENTICATION (MFA)

- MFA requires two or more forms of verification (something you know, something you have, something you are).
- Common methods: authenticator app, SMS code, hardware token, biometric.
- MFA greatly reduces the risk of unauthorized access even if your password is stolen.
- Always enable MFA, especially for cloud accounts and administrator access.



**MFA
ADDS AN EXTRA
LAYER OF SECURITY**

3. API KEYS

- API keys are used by applications to authenticate with cloud services.
- They identify the application making the request.
- Keep API keys secure and do not hardcode them in code repositories.
- Use key rotation and least privilege.



**USE API KEYS
SECURELY**

4. ACCESS KEYS

- Access keys (e.g. AWS Access Key ID and Secret Access Key) are used for programmatic access.
- They should be treated like passwords.
- Avoid long-lived access keys.
- Use IAM roles or temporary credentials when possible.
- Rotate and delete unused access keys.



**DO NOT SHARE
OR HARD CODE
ACCESS KEYS**

5. TOKENS

- Tokens are temporary credentials used to authenticate and authorize access.
- Examples: OAuth tokens, session tokens, OIDC tokens, JWT (JSON Web Tokens).
- Tokens usually expire and are safer than long-lived credentials.
- Store tokens securely and never expose them in client-side code.



**TOKENS
PROVIDE
TEMPORARY
ACCESS**

6. SSH KEYS

- SSH keys are used to securely access Linux servers and cloud instances.
- SSH keys use a public key and a private key.
- Keep your private key secure and never share it.
- Use SSH key passphrases for additional protection.
- Regularly rotate and remove unused keys.



**SSH KEYS
ENABLE SECURE
SERVER ACCESS**

7. LONG-LIVED vs TEMPORARY CREDENTIALS

- Long-lived credentials stay valid for a long time (higher risk if leaked).
- Temporary credentials expire automatically (more secure).
- Use short-lived, temporary credentials and IAM roles whenever possible.
- Avoid using long-lived credentials in automation and CI/CD pipelines.



**LONG-LIVED
HIGHER RISK**



**TEMPORARY
MORE SECURE**

8. CREDENTIAL ROTATION

- Credential rotation means regularly replacing old credentials with new ones.
- Reduces the risk of credential leakage and unauthorized access.
- Set a rotation schedule for passwords, access keys, API keys and certificates.
- Use automation where possible.



**ROTATE
REGULARLY
STAY SECURE**

9. ROOT AND HIGHLY PRIVILEGED ACCOUNTS

- Root accounts have full access to your cloud environment.
- Use root accounts only when absolutely necessary.
- Enable MFA on root accounts.
- Create and use administrative roles instead of using root.
- Monitor and audit all activity from highly privileged accounts.



**PROTECT
ROOT ACCOUNTS**

10. WHAT HAPPENS WHEN CREDENTIALS LEAK?

- Attackers can gain unauthorized access to your cloud environment.
- They can steal data, delete resources, launch new resources and increase costs.
- It can lead to service outages, data breaches and financial loss.
- Detect and respond quickly, rotate credentials and review access.



**A LEAKED
CREDENTIAL
CAN CAUSE
MAJOR DAMAGE**

CREDENTIAL SECURITY IS CLOUD SECURITY.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

SECRETS MANAGEMENT

KEEP YOUR SECRETS SECRET. PROTECT YOUR CLOUD.

1. WHAT QUALIFIES AS A SECRET?

- A secret is any sensitive information that should not be exposed to unauthorized users.
- Secrets are used to authenticate, authorize or access resources.
- If someone gets your secret, they can access your systems, data and services.
- Treat all secrets like keys to your cloud.



PROTECT
SENSITIVE
INFORMATION

2. COMMON TYPES OF SECRETS



Passwords
User passwords for systems and applications.



API tokens
Tokens used to access APIs and services.



Database credentials
Usernames and passwords for databases.



Private keys
Keys used for SSH, TLS or signing (e.g. .pem files).



Certificates
TLS/SSL certificates and private certificate keys.

3. WHY HARDCODED SECRETS ARE DANGEROUS

- Hardcoded secrets in code can be easily found by attackers.
- They can be exposed in Git repositories, logs, error messages and container images.
- Once exposed, attackers can access your cloud environment, steal data or take full control.
- Hardcoded secrets are one of the top causes of cloud security breaches.



NEVER
HARDCODE
SECRETS

4. ENVIRONMENT VARIABLES

- Environment variables can be used to store secrets instead of hardcoding them in code.
- They are better than hardcoding, but still not fully secure.
- Environment variables can be exposed in logs, process lists or misconfigured systems.
- Use environment variables for development and simple use cases, but prefer a dedicated secret manager in production.



BETTER THAN
HARDCODING
BUT STILL
NOT FULLY
SECURE

5. SECRET-MANAGEMENT SERVICES

- Cloud providers offer dedicated secret-management services.
- Examples:
 - AWS Secrets Manager
 - Azure Key Vault
 - Google Cloud Secret Manager
- These services securely store, manage and provide access to secrets.
- They support encryption, access control, auditing and rotation.



AWS
Secrets Manager
Azure
Key Vault
Google Cloud
Secret Manager

USE A SECRET
MANAGER FOR
PRODUCTION

6. ROTATION

- Secret rotation means regularly replacing old secrets with new ones.
- Rotation reduces the risk if a secret is compromised.
- Automate rotation wherever possible.
- Examples: rotating database credentials, API keys and access keys.
- Set a rotation schedule (e.g. every 30, 60 or 90 days).



ROTATE
REGULARLY
REDUCE RISK

7. GIT REPOSITORY SECRET EXPOSURE

- Secrets can be accidentally committed to Git repositories.
- Once pushed, they can be accessed by anyone with repository access (even if later deleted).
- Attackers use automated tools to scan public repositories for secrets.
- Do not commit secrets, even in private repositories.
- Use .gitignore, pre-commit hooks and secret-scanning tools.



KEEP SECRETS
OUT OF GIT

8. BEGINNER MISTAKE

Hardcoding a password in source code and pushing to GitHub.



Application



Password
inside source
code



GitHub



WRONG APPROACH

Exposes your secret to anyone who can access the repository.

9. BETTER APPROACH

Use a secret manager and give temporary authorized access.



Application



Secret
Manager



Temporary
authorized
access



RIGHT APPROACH

Keeps your secret secure and provides controlled, temporary access.

10. KEY TAKEAWAYS



- Secrets include passwords, API tokens, database credentials, private keys and certificates.
- Never hardcode secrets in code.
- Environment variables are better but still not fully secure.
- Use a secret-management service for production.
- Rotate secrets regularly.
- Prevent secret exposure in Git repositories.
- Use least privilege to access secrets.
- Treat secrets as critical assets.
- A leaked secret can lead to full account compromise.

SECURE
SECRETS
SECURE
YOUR CLOUD

SAME PRINCIPLES. A MORE SECURE YOU.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

CLOUD NETWORKING SECURITY FUNDAMENTALS

SECURE NETWORKS. SECURE APPLICATIONS. A STRONGER CLOUD.

1. VPC AND VIRTUAL NETWORK CONCEPTS

- A VPC (Virtual Private Cloud) is a logically isolated network in the cloud.
- It allows you to define your own IP address range, subnets, routing and security controls.
- It works like a private data center inside the cloud.
- You can create multiple VPCs and connect them if needed.



YOUR OWN
PRIVATE NETWORK
IN THE CLOUD

2. PUBLIC vs PRIVATE IP ADDRESSES

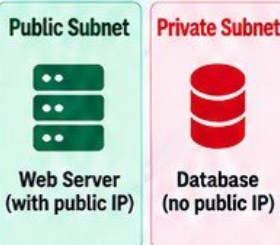
- Public IP:** reachable from the internet. Used for resources like load balancers, bastion hosts or public web servers.
- Private IP:** only reachable within the VPC. Used for internal resources like application servers and databases.
- Public IPs are like your home address (everyone can reach you).
- Private IPs are like your internal office address (only people inside can reach you).

PUBLIC IP
Reachable from the internet
e.g. 203.0.113.10

PRIVATE IP
Only reachable within the VPC
e.g. 10.0.1.10

3. PUBLIC AND PRIVATE SUBNETS

- A subnet is a smaller range of IP addresses inside a VPC.
- Public subnet:** has a route to the internet (usually through an Internet Gateway).
- Private subnet:** does not have a direct route to the internet.
- Place internet-facing resources in public subnets and sensitive resources (like databases) in private subnets.



4. PORTS AND PROTOCOLS

- Ports are communication endpoints used by applications.
- Common protocols: TCP and UDP.
- Examples:
 - 22 SSH (TCP)
 - 80 HTTP (TCP)
 - 443 HTTPS (TCP)
 - 53 DNS (TCP/UDP)
 - 3306 MySQL (TCP)
- Only open the ports you need. Close unnecessary ports.

PORT	PROTOCOL	USE
22	TCP	SSH
80	TCP	HTTP
443	TCP	HTTPS
53	TCP/UDP	DNS
3306	TCP	MySQL
5432	TCP	PostgreSQL

OPEN ONLY WHAT YOU NEED.
CLOSE THE REST.

5. ROUTING

- Routing determines where network traffic goes.
- Route tables contain rules that direct traffic to different destinations.
- Common routes:
 - Local (to other subnets in the VPC)
 - Internet Gateway (to the internet)
 - NAT Gateway (for private subnets to access the internet)
 - Peering or VPN (to connect to other networks)

Route Table	
Destination	Target
10.0.0.0/16	local
0.0.0.0/0	igw-xxxx
10.1.0.0/16	peering-xxxx

ROUTES CONTROL
THE FLOW OF TRAFFIC.

6. SECURITY GROUPS

- Security groups act as virtual firewalls at the instance level.
- They control inbound and outbound traffic.
- They are stateful (return traffic is automatically allowed).
- You can allow or deny traffic based on IP address, port and protocol.
- Use the principle of least privilege.



EXAMPLE SECURITY GROUP

- ✓ Allow 22 (SSH) from your IP
- ✓ Allow 443 (HTTPS) from anywhere
- ✗ Deny all other inbound traffic

7. NETWORK ACLS

- Network ACLs (NACLs) are firewalls at the subnet level.
- They control inbound and outbound traffic for the entire subnet.
- They are stateless (you must allow both inbound and outbound traffic).
- Use NACLs for an additional layer of security, not as a replacement for security groups.

NACL RULES (EXAMPLE)				
Rule	Type	Port	Source	Action
100	Inbound	443	0.0.0.0/0	Allow
110	Inbound	All	0.0.0.0/0	Deny
100	Outbound	All	0.0.0.0/0	Allow

NACLs ADD AN EXTRA LAYER
OF NETWORK PROTECTION.

8. FIREWALLS

- Firewalls can be used at different layers (network, host or application).
- In the cloud, this can include:
 - Security groups
 - Network ACLs
 - Managed firewalls (e.g. AWS Network Firewall, Azure Firewall)
 - Web Application Firewall (WAF)
- Firewalls help block malicious traffic and enforce security policies.



BLOCK THREATS.
ALLOW BUSINESS
TRAFFIC.

9. NETWORK SEGMENTATION

- Network segmentation means dividing your network into smaller, isolated parts.
- It limits the blast radius if a resource is compromised.
- Use separate subnets for web, application and database tiers.
- Apply different security rules to each segment.
- It improves security, compliance and control.



SEGMENT YOUR NETWORK.
REDUCE RISK.

10. INBOUND vs OUTBOUND TRAFFIC

- INBOUND TRAFFIC** ↓

 - Traffic coming into your resource (e.g. from the internet).
 - Control what can access your resources.
 - Allow only trusted sources.
- OUTBOUND TRAFFIC** ↑

 - Traffic going out from your resource (e.g. to the internet or other services).
 - Restrict unnecessary outbound traffic.
 - Helps prevent data exfiltration and malware communication.

MONITOR AND CONTROL BOTH INBOUND AND OUTBOUND TRAFFIC.

A WELL-DESIGNED NETWORK IS A MORE SECURE CLOUD.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

PROTECTING INTERNET-FACING RESOURCES

REDUCE EXPOSURE. BLOCK THREATS. KEEP YOUR CLOUD SECURE.

1. WHAT "INTERNET-FACING" MEANS

- An internet-facing resource is directly accessible from the public internet.
- It has a public IP address or is reachable through a public service (e.g. load balancer, domain name).
- Examples: web servers, APIs, public databases, remote access servers.
- Only expose resources to the internet when necessary.



IF IT IS ON THE
INTERNET, ANYONE
CAN TRY TO ACCESS IT.

2. PUBLIC IP EXPOSURE

- A public IP address is reachable from anywhere on the internet.
- Resources with public IPs can be scanned and attacked.
- Many cloud resources get public IPs by default if not configured properly.
- Use private IPs for internal resources whenever possible.
- Regularly review and remove unnecessary public IPs.



ONLY USE PUBLIC IPs
WHEN NECESSARY.

3. OPEN PORTS

- Ports are entry points to your resource.
- Open ports allow network traffic to reach specific services.
- Common ports: 22 (SSH), 80 (HTTP), 443 (HTTPS), 3389 (RDP).
- Only open the ports you need.
- Close unused ports.



OPEN PORTS
INCREASE YOUR
ATTACK SURFACE.

4. WHY 0.0.0.0/0 MATTERS

- 0.0.0.0/0 means "allow from any IP address" (everywhere on the internet).
- It is commonly used in security groups, firewall rules and ACLs.
- This makes your resource accessible from any country, by anyone.
- Only use 0.0.0.0/0 when absolutely necessary (e.g. for a public web server).
- Use specific IP ranges instead whenever possible.



0.0.0.0/0
ALLOWS EVERYONE.
USE IT CAREFULLY.

5. SSH AND RDP EXPOSURE

- SSH (port 22) and RDP (port 3389) are commonly targeted by attackers.
- Never expose SSH or RDP to the entire internet (0.0.0.0/0).
- Use a VPN, bastion host or just-in-time access instead.
- Restrict access to known IP addresses.
- Use key-based authentication for SSH and strong authentication for RDP.



DO NOT EXPOSE SSH
OR RDP TO EVERYONE.

6. LOAD BALANCERS

- A load balancer sits in front of your applications and distributes traffic across multiple servers.
- It can provide a single public endpoint for your application.
- Use a load balancer to improve availability, scalability and security.
- Combine with security groups and WAF for better protection.



USE LOAD BALANCERS
TO MANAGE AND PROTECT
INCOMING TRAFFIC.

7. WEB APPLICATION FIREWALL (WAF)

- A WAF filters and monitors HTTP/HTTPS traffic to protect your web applications.
- It helps block common attacks like: SQL injection, XSS and malicious bots.
- WAF can be configured with managed rules and custom rules.
- Use WAF with a load balancer or CDN for public web applications.



WAF HELPS BLOCK
MALICIOUS TRAFFIC
BEFORE IT REACHES
YOUR APPLICATION.

8. HTTPS AND TLS

- HTTPS encrypts data between users and your application.
- TLS (Transport Layer Security) provides the encryption.
- Always use HTTPS for internet-facing applications.
- Use trusted certificates (e.g. from AWS ACM, Let's Encrypt or a CA).
- Redirect HTTP to HTTPS.



ENCRYPT DATA.
PROTECT YOUR USERS.

9. ADMINISTRATIVE INTERFACES

- Administrative interfaces (e.g. cloud consoles, management dashboards, databases) are high-value targets.
- Do not expose admin interfaces to the internet.
- Use VPN, private networks or IP allow lists.
- Enable MFA for all admin access.
- Monitor and log all administrative activity.



KEEP ADMIN INTERFACES
PRIVATE AND PROTECTED.

10. REDUCING ATTACK SURFACE

- Only expose what you need.
- Use private subnets for internal resources.
- Close unused ports and services.
- Remove unnecessary public IPs.
- Use security groups, NACLs and firewalls.
- Apply least privilege.
- Regularly review and audit your configuration.



LESS EXPOSURE.
LESS RISK.
A MORE SECURE CLOUD.

SECURE TODAY. A SAFER TOMORROW.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

PROTECTING CLOUD DATA

YOUR DATA IS YOUR BUSINESS. PROTECT IT AT EVERY LAYER.

1. DATA AT REST

- Data at rest is data stored on disk in the cloud (e.g. databases, object storage, file systems, backups).
- Examples: Amazon S3, EBS, RDS, DynamoDB, files, logs, snapshots.
- Always enable encryption for data at rest.
- Control who can access the data using IAM and policies.



ENCRYPT
YOUR STORED
DATA

2. DATA IN TRANSIT

- Data in transit is data moving between systems over a network (e.g. your browser to a website, application to database).
- It can be intercepted if not encrypted.
- Use secure protocols like HTTPS, TLS, SSH, and VPN.
- Always encrypt data in transit, especially over the internet.



KEEP DATA ENCRYPTED
FROM END TO END

3. ENCRYPTION

- Encryption converts readable data (plaintext) into unreadable data (ciphertext).
- It protects data from being read by unauthorized users.
- Use encryption for data at rest and in transit.
- Modern cloud providers make encryption easy to enable.



Plaintext



7f3a9c
b2e1d4
9c8f0a

Ciphertext

ENCRYPTION KEEPS YOUR DATA SAFE

4. PLAINTEXT vs CIPHERTEXT

- Plaintext: original, readable data (e.g. Hello, VERIQTA).
- Ciphertext: encrypted, unreadable data (e.g. H3f9@kL2...).
- Only authorized users with the decryption key can convert ciphertext back to plaintext.

Plaintext → Hello, VERIQTA
Ciphertext → H3f9@kL2...

KEEP DATA
ENCRYPTED.
DON'T LEAVE
SENSITIVE DATA
IN PLAINTEXT.

5. TLS

- TLS (Transport Layer Security) secures communication over a network.
- It encrypts the data and prevents eavesdropping and tampering.
- Used by HTTPS, email, APIs and many cloud services.
- Always use the latest TLS version (e.g. TLS 1.2 or 1.3).



TLS PROTECTS
YOUR DATA
IN TRANSIT

6. HTTPS

- HTTPS is HTTP over TLS.
- It encrypts the communication between your browser and the server.
- Always use HTTPS for websites, APIs and any internet-facing services.
- Look for the padlock icon in your browser.



USE HTTPS
ALWAYS

7. ENCRYPTION KEYS

- Encryption keys are used to encrypt and decrypt data.
- Keep keys secure and restrict access.
- Never hardcode keys in your application code.
- Use a key management service (KMS) in the cloud.



PROTECT
YOUR KEYS
LIKE YOUR DATA

8. KEY MANAGEMENT SERVICE (KMS)

- KMS is a managed service that helps you create, store and manage encryption keys.
- Examples: AWS KMS, Azure Key Vault, Google Cloud KMS.
- It provides secure key storage, access control, auditing and key rotation.



USE KMS
TO SECURELY
MANAGE
YOUR KEYS

9. KEY ROTATION

- Key rotation means regularly replacing old encryption keys with new ones.
- It reduces the risk if a key is compromised.
- Use automatic key rotation where possible.
- Set a rotation schedule (e.g. every 90 days or 1 year).



ROTATE KEYS
REGULARLY
REDUCE RISK

10. BACKUPS

- Backups protect your data from accidental deletion, corruption and ransomware.
- Encrypt your backups.
- Store backups in a different region or account.
- Regularly test your backups to make sure they can be restored.



BACKUP
ENCRYPT
TEST
BE PREPARED

11. ACCESS CONTROL

- Use IAM, roles and policies to control who can access your data.
- Give the minimum permissions (least privilege).
- Regularly review access.
- Remove unnecessary or inactive access.



RIGHT PEOPLE
RIGHT ACCESS
RIGHT DATA

12. WHY ENCRYPTION ALONE IS NOT ENOUGH

- Encryption protects data, but it does not stop unauthorized access.
- Attackers can still delete, modify or misuse encrypted data if they have access.
- You also need strong identity and access control, monitoring, secure configurations, network security and regular backups.
- Use a layered security approach.



ENCRYPTION
IS IMPORTANT
BUT NOT
ENOUGH

SECURE DATA. BUILD TRUST. GO FURTHER WITH VERIQTA.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

SECURING CLOUD STORAGE

STORE SMART. PROTECT BETTER. KEEP YOUR DATA SAFE.

1. OBJECT STORAGE

- Object storage is used to store large amounts of unstructured data (e.g. files, backups, images, logs).
- Examples: AWS S3, Azure Blob Storage, Google Cloud Storage.
- Data is stored as objects, not in a traditional file system.
- It is highly scalable, durable and cost-effective.



IDEAL FOR
FILES, BACKUPS,
LOGS, AND
LARGE DATASETS.

2. BUCKETS AND CONTAINERS

- In object storage, data is stored in buckets (AWS), containers (Azure) or buckets (GCP).
- A bucket is like a folder that holds your objects (files).
- Each object has a unique key (name).
- You can organize objects using prefixes (similar to folders).



my-bucket
images/
logs/
backups/
documents/

ORGANIZE YOUR DATA
FOR EASY MANAGEMENT.

3. PRIVATE vs PUBLIC STORAGE

- Private storage: only authorized users can access the data.
- Public storage: accessible by anyone on the internet.
- By default, buckets should be private.
- Only make data public when absolutely necessary (e.g. for a public website).



PRIVATE
(Only authorized access)



PUBLIC
(Accessible by anyone)

KEEP YOUR BUCKETS PRIVATE
BY DEFAULT.

4. BUCKET POLICIES

- Bucket policies define who can access your bucket and what actions they can perform.
- Policies use JSON format.
- You can allow or deny access based on:
 - IAM users or roles
 - IP addresses
 - Specific resources
 - Conditions (e.g. HTTPS only)
- Always follow the principle of least privilege.



USE BUCKET POLICIES
TO CONTROL ACCESS
PRECISELY.

5. PUBLIC ACCESS CONTROLS

- Cloud providers allow you to block public access at the account, bucket or container level.
- Use settings like:
 - Block public access (AWS S3)
 - Public access settings (Azure)
 - Public access prevention (GCP)
- Regularly review and test your public access settings.



BLOCK PUBLIC
ACCESS UNLESS
YOU REALLY
NEED IT.

6. ENCRYPTION

- Encrypt your data at rest to protect it from unauthorized access.
- Use cloud provider managed keys or your own keys (e.g. AWS KMS, Azure Key Vault, Google Cloud KMS).
- You can also encrypt data in transit using TLS (covered in an earlier page).
- Encryption helps keep your data safe even if it is accidentally exposed.



ENCRYPT YOUR
DATA AT REST
FOR EXTRA
PROTECTION.

7. VERSIONING

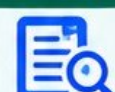
- Versioning keeps multiple versions of an object.
- It helps you recover from accidental deletions, overwrites or ransomware.
- You can restore previous versions when needed.



ENABLE VERSIONING
TO PROTECT AGAINST
ACCIDENTAL LOSS.

8. LOGGING

- Enable access logs to track who accessed your bucket and what actions were performed.
- Logs help with security monitoring, auditing and incident investigation.
- You can send logs to a separate bucket or to a logging service (e.g. CloudTrail, Azure Monitor, Cloud Logging).



LOG AND MONITOR
ACCESS TO DETECT
SUSPICIOUS ACTIVITY.

9. BACKUP AND RECOVERY

- Keep backups of important data.
- Use versioning, cross-region replication or lifecycle policies.
- Test your recovery process regularly.
- Have a clear plan for data restoration in case of accidental deletion or corruption.



PLAN AHEAD.
TEST YOUR
RECOVERY.

10. ACCIDENTAL DATA EXPOSURE

- Misconfigured buckets are one of the biggest causes of data leaks in the cloud.
- Sensitive data (e.g. company files, customer data, databases) can be exposed to the internet by mistake.
- Regularly review your bucket settings, permissions and public access controls.



ALWAYS CHECK
YOUR CONFIGURATION.
PREVENT DATA LEAKS.

11. FAILURE SCENARIO



Private company file
(e.g. financial report)



Misconfigured bucket
(with public access)



Public internet access
(anyone can download it)

REAL INCIDENTS HAVE EXPOSED:

- Company financial data
- Customer personal information
- Internal documents
- Source code and API keys



A SIMPLE
MISCONFIGURATION
CAN LEAD TO A
MAJOR DATA BREACH.

PROTECT YOUR DATA TODAY. A SAFER TOMORROW WITH VERIQTA.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

SECURING VIRTUAL MACHINES

HARDEN YOUR VMs. REDUCE RISK. KEEP YOUR CLOUD SECURE.

1. VIRTUAL MACHINE ATTACK SURFACE

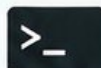
- A virtual machine (VM) can be attacked through its network, operating system, applications and configurations.
- Exposed ports, weak credentials, unpatched software and misconfigurations increase risk.
- Attackers can use the VM to gain access to other resources in your cloud environment.



THE BIGGER
THE ATTACK SURFACE,
THE HIGHER
THE RISK.

2. SSH AND RDP

- SSH (port 22) is used for Linux VMs. RDP (port 3389) is used for Windows VMs.
- Never expose SSH or RDP to the entire internet (0.0.0.0/0).
- Use key-based authentication for SSH.
- Use strong passwords and MFA for RDP when possible.
- Restrict access to known IP addresses or use a VPN or bastion host.



SSH



RDP

DON'T EXPOSE
SSH OR RDP
TO EVERYONE.

3. OPERATING-SYSTEM HARDENING

- Remove unnecessary software and services.
- Disable unused accounts.
- Configure secure system settings (e.g. disable password login, set secure file permissions).
- Use benchmarks like CIS (for Linux and Windows).
- Regularly review and harden your OS configuration.



A SECURE OS
REDUCES
YOUR RISK.

4. PATCH MANAGEMENT

- Regularly install security patches for the operating system and installed software.
- Patches fix known vulnerabilities that attackers can exploit.
- Use automated patch management where possible (e.g. AWS SSM, Azure Update Management).
- Test patches in non-production environments before applying in production.



KEEP YOUR
SYSTEMS PATCHED
TO CLOSE
KNOWN RISKS.

5. PACKAGE VULNERABILITIES

- Applications and packages installed on your VM can have vulnerabilities.
- Regularly scan for vulnerabilities (e.g. using tools like Trivy, Lynis or OS package managers).
- Remove unused packages.
- Keep packages up to date.
- Monitor security advisories for critical updates.



OUTDATED
PACKAGES
ARE A COMMON
ENTRY POINT
FOR ATTACKERS.

6. UNNECESSARY SERVICES

- Disable or uninstall services that you do not use.
- Fewer services mean fewer opportunities for attackers.
- Common unnecessary services: FTP, Telnet, old database services, or demo applications.
- Regularly audit running services and open ports.



ONLY RUN
WHAT YOU NEED.

7. HOST FIREWALLS

- Use the operating system firewall to control inbound and outbound traffic (e.g. iptables, firewalld, Windows Firewall).
- Allow only necessary ports and services.
- Combine host firewalls with cloud security groups for defense in depth.
- Regularly review firewall rules.



BLOCK
UNWANTED TRAFFIC.
ALLOW ONLY
WHAT YOU NEED.

8. SECURE IMAGES

- Use official and trusted images from your cloud provider or official sources.
- Keep your base images updated and hardened.
- Remove unnecessary tools and default accounts.
- Consider using a golden image or infrastructure as code (IaC) to maintain consistent security.



START WITH
A SECURE IMAGE
TO REDUCE RISK
FROM DAY ONE.

9. INSTANCE ROLES

- Use instance roles (e.g. AWS IAM roles, Azure managed identities) to give your VM temporary access to cloud resources.
- Avoid embedding long-lived access keys in your VM.
- Follow the principle of least privilege.



USE INSTANCE
ROLES INSTEAD
OF ACCESS KEYS.

10. METADATA SERVICES

- Cloud providers offer metadata services (e.g. AWS IMDS, Azure Instance Metadata Service) to provide instance information and temporary credentials.
- Protect metadata services from unauthorized access.
- Use IMDSv2 (on AWS) to prevent server-side request forgery (SSRF) attacks.
- Do not expose metadata endpoints to untrusted applications.



PROTECT
METADATA SERVICES
THEY CONTAIN
SENSITIVE
INFORMATION.

11. LOGGING

- Enable and collect logs from your VMs (e.g. system logs, authentication logs, application logs).
- Send logs to a centralized logging service (e.g. CloudWatch, Azure Monitor, ELK stack).
- Monitor logs for suspicious activity.
- Keep logs for incident investigation and compliance.



LOGS HELP YOU
DETECT, INVESTIGATE
AND RESPOND
TO THREATS.

12. AVOIDING UNNECESSARY ROOT ACCESS

- Do not use the root account for daily tasks.
- Use a standard user account with sudo (Linux) or an administrator account (Windows).
- Restrict and monitor the use of root and privileged accounts.
- Enable MFA for privileged accounts.



USE LEAST
PRIVILEGE.
AVOID ROOT
ACCESS.

SECURE VMs. STRONGER INFRASTRUCTURE. A SAFER TOMORROW.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

CONTAINER SECURITY FOR BEGINNERS

BUILD SECURE CONTAINERS. RUN SAFER APPLICATIONS. PROTECT YOUR CLOUD.

1. IMAGE vs CONTAINER

- An image is a read-only template that contains the application, dependencies and configuration.
- A container is a running instance of an image.
- You can create multiple containers from the same image.



IMAGE IS THE BLUEPRINT.
CONTAINER IS THE RUNNING ENVIRONMENT.

2. WHY CONTAINERS INTRODUCE SECURITY CONCERNS

- Containers share the host OS kernel.
- They can run untrusted code.
- Vulnerabilities in images or applications can be exploited.
- A compromised container can affect the host or other containers if not properly isolated.
- Containers are often deployed quickly, which can lead to skipped security checks.



SPEED AND PORTABILITY ARE BENEFITS,
BUT THEY ALSO INTRODUCE RISK.

3. TRUSTED BASE IMAGES

- Use official and trusted base images (e.g. from Docker Hub, official cloud providers).
- Prefer minimal images (e.g. distroless, alpine) to reduce the attack surface.
- Avoid unknown or unmaintained images.
- Regularly check for updates and security patches.



START WITH A TRUSTED, MINIMAL,
UP-TO-DATE BASE IMAGE.

4. VULNERABLE DEPENDENCIES

- Images include OS packages and application dependencies.
- Vulnerabilities in these dependencies can be exploited by attackers.
- Regularly scan and update dependencies.
- Remove unnecessary packages to reduce risk.



LESS PACKAGES = SMALLER ATTACK SURFACE.

5. IMAGE SCANNING

- Use image scanning tools to detect known vulnerabilities in your images.
- Examples: Trivy, Gripe, Docker Scout, Snyk.
- Scan images in your CI/CD pipeline before deployment.
- Fail builds if critical vulnerabilities are found.



SCAN EARLY. FIX ISSUES BEFORE
YOUR IMAGES REACH PRODUCTION.

6. PRIVATE REGISTRIES

- Use private registries to store your container images.
- Limit access to authorized users and systems only.
- Examples: Amazon ECR, Azure Container Registry, Google Artifact Registry, JFrog Artifactory, Harbor.
- Enable authentication and access control (IAM/RBAC).



KEEP YOUR IMAGES PRIVATE
AND ACCESS CONTROLLED.

7. CONTAINER SECRETS

- Do not hardcode secrets in your images.
- Use secret management tools (e.g. Docker secrets, HashiCorp Vault, AWS Secrets Manager).
- Pass secrets at runtime, not build time.
- Avoid environment variables for sensitive data when possible.



SECRETS BELONG IN A SECRET MANAGER,
NOT IN YOUR IMAGE.

8. RUNNING AS NON-ROOT

- Do not run containers as the root user.
- Use a non-root user inside the container.
- This limits the damage if the container is compromised.
- Set the user in your Dockerfile (e.g. USER appuser).



Good practice
FROM alpine
RUN adduser -D appuser
USER appuser

RUN AS A NON-ROOT USER
BY DEFAULT.

9. PRIVILEGED CONTAINERS

- Avoid running privileged containers (privileged).
- Privileged containers have extended access to the host and can be dangerous if compromised.
- Only use privileged mode when absolutely necessary.
- Consider using specific capabilities instead of full privileged mode (e.g. --cap-add).



DO NOT USE PRIVILEGED CONTAINERS
UNLESS YOU REALLY NEED TO.

10. RUNTIME SECURITY

- Use runtime security tools to monitor container behavior.
- Detect and block suspicious activity.
- Examples: Falco, Aqua, Sysdig.
- Apply network policies to restrict container communication.
- Use read-only file systems where possible.



MONITOR. DETECT. RESPOND.
KEEP YOUR CONTAINERS SAFE AT RUNTIME.

11. KEEPING IMAGES UPDATED

- Regularly rebuild images with the latest security patches.
- Automate image updates using CI/CD pipelines.
- Remove old and unused images.
- Subscribe to security advisories for your base images and dependencies.
- Use tools like Dependabot or Renovate to track updates.
- Test updated images before deploying to production.



KEEP YOUR IMAGES
UP TO DATE
TO STAY SECURE.

SECURE CONTAINERS TODAY. STRONGER APPLICATIONS TOMORROW.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

KUBERNETES SECURITY FUNDAMENTALS



SECURE YOUR CLUSTER. PROTECT YOUR APPLICATIONS. BUILD FOR PRODUCTION.

1. KUBERNETES SECURITY BOUNDARIES

- Kubernetes has multiple security boundaries: cluster, namespace, workload (pod), and container.
- Each layer provides a different level of isolation and control.
- Security is shared responsibility between the platform team and application teams.



UNDERSTAND THE BOUNDARIES
TO APPLY THE RIGHT SECURITY CONTROLS.

2. CLUSTER ACCESS

- Cluster access is usually managed using kubeconfig, certificates or cloud IAM (e.g. AWS IAM, Azure AD).
- Limit who can access the cluster.
- Use MFA and short-lived credentials where possible.
- Regularly review and remove unused access.



ONLY TRUSTED USERS SHOULD HAVE
CLUSTER ACCESS.

3. KUBERNETES RBAC

- RBAC (Role-Based Access Control) controls what users, groups and ServiceAccounts can do.
- Use Roles (namespace level) and ClusterRoles (cluster level).
- Follow the principle of least privilege.
- Avoid giving cluster-admin unless absolutely necessary.
- Regularly review RBAC permissions.



GIVE ONLY THE PERMISSIONS NEEDED.
NO MORE. NO LESS.

4. SERVICEACCOUNTS

- ServiceAccounts provide an identity for pods to access the Kubernetes API and other resources.
- Each namespace has a default ServiceAccount (avoid using it).
- Create dedicated ServiceAccounts with minimal permissions.
- Use automatically mounted tokens and avoid long-lived credentials.



USE DEDICATED SERVICEACCOUNTS
WITH LEAST PRIVILEGE.

5. NAMESPACES

- Namespaces provide logical isolation within a cluster.
- Use namespaces to separate teams, environments (dev, staging, prod) and workloads.
- Apply resource quotas and limit ranges.
- Combine with RBAC and NetworkPolicies for stronger isolation.



USE NAMESPACES TO ISOLATE
AND ORGANIZE YOUR WORKLOADS.

6. SECRETS

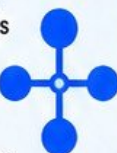
- Kubernetes Secrets store sensitive data such as passwords, API keys and certificates.
- Secrets are base64 encoded (not encrypted by default).
- Restrict access using RBAC.
- Consider using external secret management tools (e.g. AWS Secrets Manager, HashiCorp Vault).
- Avoid hardcoding secrets in images or manifests.



KEEP SECRETS SECURE AND
LIMIT WHO CAN ACCESS THEM.

7. NETWORKPOLICIES

- NetworkPolicies control network traffic between pods, namespaces and external services.
- Use them to allow only necessary communication.
- Follow a default deny approach and explicitly allow required traffic.
- Helps reduce the impact of a compromised pod.



ALLOW ONLY THE TRAFFIC YOU NEED.
BLOCK EVERYTHING ELSE.

8. POD SECURITY STANDARDS

- Pod Security Standards (PSS) define different security levels: Privileged, Baseline and Restricted.
- Restricted is the recommended level for most workloads.
- Helps prevent dangerous configurations (e.g. running as root, privileged containers, hostPath).
- Enforce using namespace labels or admission controllers.



USE RESTRICTED POLICIES
FOR BETTER SECURITY.

9. PRIVILEGED CONTAINERS

- Privileged containers have extended access to the host and can be dangerous if compromised.
- Avoid using privileged containers unless absolutely necessary.
- They can access host devices, kernel features and sensitive data on the node.



DO NOT USE PRIVILEGED CONTAINERS
UNLESS YOU REALLY NEED TO.

10. API SERVER PROTECTION

- The Kubernetes API server is the control plane and must be protected.
- Use TLS and strong authentication (e.g. client certificates, IAM).
- Restrict access using firewalls, private networks or a bastion host.
- Regularly audit and monitor API server access.



PROTECT THE API SERVER. IT CONTROLS YOUR CLUSTER.

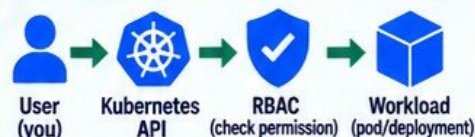
11. ADMISSION CONTROLS

- Admission controls validate and can modify requests before they are persisted.
- Use built-in admission controllers (e.g. Pod Security, Namespace Lifecycle, ResourceQuota).
- You can also use tools like OPA/Gatekeeper for custom policies.
- Helps enforce security and compliance rules across the cluster.



USE ADMISSION CONTROLS TO ENFORCE SECURITY POLICIES.

12. SIMPLE MENTAL MODEL



User → Kubernetes API → RBAC → Workload
A request goes to the API server, RBAC checks permissions, and if allowed, it can create or manage the workload.

SECURE KUBERNETES TODAY. STRONGER APPLICATIONS TOMORROW.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

INFRASTRUCTURE AS CODE SECURITY

WRITE SECURE INFRASTRUCTURE. PREVENT RISKS BEFORE DEPLOYMENT.

1. WHAT IS INFRASTRUCTURE AS CODE (IaC)?

- Infrastructure as Code (IaC) means managing and provisioning infrastructure using code instead of manual processes.
- You define your cloud resources in configuration files (e.g. Terraform) and deploy them automatically.
- IaC makes infrastructure repeatable, version controlled and easier to manage.



CODE
YOUR
INFRASTRUCTURE
LIKE
APPLICATIONS.

2. TERRAFORM AS AN EXAMPLE

- Terraform is a popular IaC tool created by HashiCorp.
- You write configuration files in HashiCorp Configuration Language (HCL).
- Terraform can provision resources in AWS, Azure, GCP and many other platforms.



```
resource "aws_instance" "web" {
  ami           = "ami-12345678"
  instance_type = "t3.micro"
}
```

TERRAFORM TURNS CODE INTO REAL
INFRASTRUCTURE.

3. WHY CONFIGURATION BECOMES SECURITY-SENSITIVE CODE

- Your IaC code defines real cloud infrastructure, so mistakes can create security risks.
- Configurations may include sensitive data, permissions, network rules and access controls.
- If an attacker gains access to your IaC code, they can understand and potentially modify your infrastructure.



TREAT YOUR IaC CODE
AS SENSITIVE.
IT CAN IMPACT YOUR
ENTIRE CLOUD ENVIRONMENT.

4. INSECURE DEFAULTS

- Cloud providers often have insecure default settings.
- Examples:
 - Public access enabled.
 - Open security group rules.
 - Excessive permissions.
 - Unencrypted resources.
- Always review and explicitly set secure configurations in your IaC code.



NEVER RELY
ON DEFAULTS.
ALWAYS
SET SECURE
VALUES.

5. HARDCODED SECRETS

- Never hardcode secrets in IaC files (e.g. passwords, API keys, tokens).
- Hardcoded secrets can be exposed in Git repositories, logs and Terraform state files.
- Use secret management solutions (e.g. AWS Secrets Manager, Azure Key Vault, HashiCorp Vault).

```
# Bad example
password = "MyP@ssw0rd123"
```



USE A SECRETS MANAGER.
DO NOT HARDCODE SECRETS.

6. PUBLIC RESOURCE MISTAKES

- Accidentally making resources public is a common IaC mistake.
- Examples:
 - Public S3 buckets.
 - Open security group rules (0.0.0.0/0).
 - Public databases.
 - Public load balancers.
- Always validate network and access settings before deployment.



LEAST PRIVILEGE.
KEEP RESOURCES
PRIVATE UNLESS
PUBLIC ACCESS
IS REQUIRED.

7. TERRAFORM STATE SECURITY

- The Terraform state file stores the current state of your infrastructure.
- It can contain sensitive information (e.g. resource IDs, IP addresses, secrets).
- Store state files securely using remote backends (e.g. S3 with encryption and access control).
- Do not commit state files to your Git repository.



PROTECT YOUR STATE FILE.
USE REMOTE STORAGE WITH ENCRYPTION.

8. CODE REVIEW

- Always review IaC code before merging and deploying.
- Look for security issues such as:
 - Public resources.
 - Excessive permissions.
 - Hardcoded secrets.
 - Insecure defaults.
- Use pull requests and peer review processes, just like application code.



A SECOND PAIR OF EYES CAN
PREVENT MAJOR SECURITY ISSUES.

9. IaC SECURITY SCANNING

- Use automated scanning tools to find security misconfigurations in your IaC code.
- Examples:
 - Checkov
 - tfsec
 - Terrascan
 - Trivy (for IaC)
- Integrate scanning into your CI/CD pipeline.



AUTOMATE SECURITY CHECKS
IN YOUR PIPELINE.

10. POLICY AS CODE

- Use policy as code to enforce security and compliance rules.
- Examples:
 - Open Policy Agent (OPA)
 - HashiCorp Sentinel
 - AWS Config Rules
- Define rules such as:
 - No public S3 buckets.
 - Use encrypted storage.
 - Restrict allowed instance types.



AUTOMATE POLICIES TO KEEP
YOUR INFRASTRUCTURE COMPLIANT.

11. CATCHING PROBLEMS BEFORE DEPLOYMENT

- Use a combination of:
 - Code review
 - Automated security scanning
 - Policy as code
 - Testing in non-production environments
- Catch and fix security issues early to reduce risk and prevent incidents in production.



FIND AND FIX SECURITY ISSUES
BEFORE YOU DEPLOY.

12. SIMPLE MENTAL MODEL

FROM CODE TO SECURE INFRASTRUCTURE



User → IaC Tool → Policies/Checks → Secure Infrastructure. Always validate, scan and enforce policies before deployment.

SECURE INFRASTRUCTURE TODAY. A SAFER TOMORROW WITH VERIQTA.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

CI/CD AND SOFTWARE SUPPLY CHAIN SECURITY

SECURE YOUR PIPELINES. PROTECT YOUR CODE. DELIVER WITH CONFIDENCE.

1. WHY PIPELINES NEED PROTECTION

- CI/CD pipelines have access to your source code, secrets, infrastructure and production environments.
- Attackers target pipelines to steal code, inject malicious code, or deploy compromised software.
- A compromised pipeline can affect your entire software supply chain.
- Protecting the pipeline helps ensure the integrity, security and trust of your software.



IF AN ATTACKER CONTROLS YOUR PIPELINE, THEY CAN CONTROL YOUR PRODUCT.

2. SOURCE CODE

- Store code in a secure Git repository (e.g. GitHub, GitLab, Bitbucket).
- Use protected branches (main).
- Require pull requests and code reviews.
- Enable branch protection rules.
- Use commit signing where possible.



PROTECT YOUR SOURCE CODE. IT IS THE START OF EVERYTHING.

3. DEPENDENCIES

- Applications depend on open source and third-party libraries.
- Vulnerable dependencies can introduce serious security risks (e.g. log4j, etc).
- Use dependency scanning tools (e.g. Snyk, Dependabot, OWASP Dependency-Check).
- Regularly update dependencies to patch known vulnerabilities.



SCAN YOUR DEPENDENCIES. DON'T IMPORT RISK.

4. BUILD SYSTEMS

- Build systems compile your code and create artifacts.
- If the build system is compromised, attackers can inject malicious code into your build.
- Use trusted build images and tools.
- Keep build environments updated and minimal.
- Monitor build logs for suspicious activity.



A COMPROMISED BUILD SYSTEM CAN PRODUCE COMPROMISED SOFTWARE.

5. CI/CD RUNNERS

- Runners execute your pipeline jobs.
- Self-hosted runners must be hardened and isolated.
- Use short-lived, ephemeral runners where possible.
- Restrict network access from runners.
- Keep runner images updated.



TREAT RUNNERS AS HIGH-RISK. LOCK THEM DOWN.

6. PIPELINE SECRETS

- Pipelines use secrets (API keys, cloud credentials, tokens, etc).
- Never hardcode secrets in code or pipeline files.
- Use a secrets manager (e.g. GitHub Secrets, GitLab CI variables, AWS Secrets Manager, HashiCorp Vault).
- Rotate secrets regularly.
- Limit secret access to only what is needed.



KEEP SECRETS SECRET. USE A SECRETS MANAGER.

7. ARTIFACT INTEGRITY

- Artifacts (binaries, container images, packages) must be trusted and unmodified.
- Use checksums, digital signatures and provenance (e.g. SLSA).
- Verify artifacts before deployment.
- Store artifacts in trusted repositories.



VERIFY YOUR ARTIFACTS. TRUST BUT ALWAYS VERIFY.

8. CONTAINER REGISTRIES

- Store container images in private registries (e.g. ECR, GCR, ACR, Docker Hub with access control).
- Scan images for vulnerabilities before pushing and before deploy.
- Use image signing (e.g. Cosign).
- Restrict access with IAM/RBAC.
- Enable image immutability where possible.



SCAN, SIGN AND RESTRICT ACCESS TO YOUR CONTAINER IMAGES.

9. DEPLOYMENT CREDENTIALS

- Pipelines use credentials to deploy to cloud and production environments.
- Use least-privilege access (e.g. IAM roles with minimal permissions).
- Avoid long-lived static credentials.
- Use short-lived credentials (e.g. OIDC, workload identity).
- Regularly review and rotate credentials.



GIVE ONLY THE MINIMUM PERMISSIONS NEEDED.

10. PROTECTED BRANCHES

- Use branch protection rules for main/production branches.
- Require pull requests and code reviews.
- Restrict who can push or merge.
- Use status checks (tests, security scans) before merge.
- Prevent force pushes and deletions on important branches.



PROTECT YOUR BRANCHES. PREVENT UNAUTHORIZED CHANGES.

11. DEPENDENCY SCANNING

- Use automated tools to scan for vulnerable dependencies.
- Examples: Snyk, Dependabot, OWASP Dependency-Check, Trivy.
- Scan during build and regularly in your repository.
- Fail the build if critical vulnerabilities are found.



FIND AND FIX VULNERABILITIES BEFORE THEY REACH PRODUCTION.

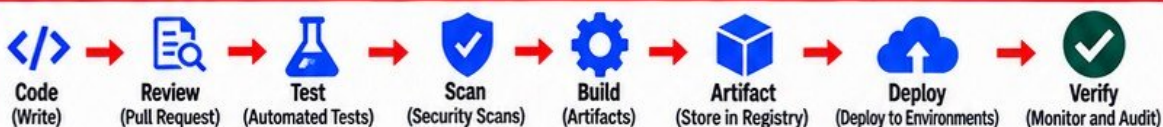
12. LEAST-PRIVILEGE PIPELINES

- Give pipelines only the permissions they need.
- Use fine-grained IAM roles and RBAC.
- Separate environments (dev, staging, prod) with different credentials.
- Use approval steps for production deployments.
- Regularly review and audit permissions.



LEAST PRIVILEGE REDUCES RISK AND LIMITS DAMAGE.

13. SECURE FLOW



A SECURE SUPPLY CHAIN DELIVERS TRUSTED SOFTWARE TO A SAFER TOMORROW.

SECURE TODAY. STRONGER SOFTWARE TOMORROW WITH VERIQTA.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

VULNERABILITIES, PATCHING, AND SECURE CONFIGURATION



FIND. FIX. HARDEN. KEEP YOUR ENVIRONMENT SECURE.

1. WHAT A VULNERABILITY IS

- A vulnerability is a weakness in software, hardware, configuration or a process that can be exploited by an attacker.
- It can allow unauthorized access, data theft, service disruption or other malicious activity.
- Vulnerabilities exist in operating systems, applications, libraries, containers, cloud services and network devices.



A VULNERABILITY IS A WEAKNESS THAT CAN BE EXPLOITED.

2. CVE BASICS

- CVE (Common Vulnerabilities and Exposures) is a public identifier for known vulnerabilities.
- Each CVE has a unique ID (e.g. CVE-2024-3094).
- CVE records include a description, severity (CVSS score), affected products and references.
- Use CVE databases (e.g. NVD) to learn more about vulnerabilities.



CVE GIVES A STANDARD NAME TO A KNOWN VULNERABILITY.

3. VULNERABILITY vs EXPLOIT

- A vulnerability is the weakness in the system.
- An exploit is the code or technique used to take advantage of a vulnerability.
- Not all vulnerabilities have public exploits.
- The risk increases when a vulnerability has a known and easy-to-use exploit.



A VULNERABILITY CAN BECOME DANGEROUS WHEN AN EXPLOIT EXISTS.

4. VULNERABILITY SCANNING

- Use scanning tools to automatically find known vulnerabilities.
- Examples: Nessus, OpenVAS, Qualys, Trivy, Snyk, AWS Inspector.
- Scan your servers, containers, images, cloud resources and applications regularly.
- Review scan reports and validate the findings.



SCAN REGULARLY TO FIND VULNERABILITIES BEFORE ATTACKERS DO.

5. PATCH MANAGEMENT

- Apply security patches to fix known vulnerabilities.
- Prioritize critical and high-severity patches.
- Test patches in a non-production environment when possible.
- Use automated patch management tools (e.g. AWS SSM, Azure Update Manager, Ansible, SCCM).
- Keep track of patch status and compliance.



PATCH EARLY. PATCH REGULARLY. REDUCE RISK.

6. CONFIGURATION WEAKNESSES

- Misconfigurations can create security risks even without software bugs.
- Examples: open ports, overly permissive access, default settings, public storage.
- Follow security best practices and hardened configuration guides (e.g. CIS Benchmarks).
- Review configurations regularly.



SECURE CONFIGURATIONS CLOSE EASY ATTACK PATHS.

7. ASSET INVENTORY

- Maintain an up-to-date inventory of all assets (servers, VMs, containers, applications, cloud resources, etc.).
- You cannot secure what you do not know exists.
- Include asset owners, environment (dev, staging, prod) and criticality.
- Use tools like AWS Config, Azure Resource Graph, CMDB or open-source asset trackers.



KNOW YOUR ASSETS. IT'S THE FIRST STEP TO SECURITY.

8. SECURITY BASELINES

- A security baseline is a minimum set of secure configuration standards.
- Use industry standards like CIS Benchmarks, NIST or your organization's policies.
- Apply baselines to operating systems, cloud services, databases, containers and network devices.
- Regularly audit systems against the baseline.



BASELINES PROVIDE A SECURE STARTING POINT.

9. CONFIGURATION DRIFT

- Configuration drift happens when systems change from the approved and secure configuration.
- It can be caused by manual changes, failed deployments or automated processes.
- Use configuration management tools (e.g. Ansible, Terraform, AWS Config) to detect and prevent drift.
- Regularly compare current settings with your baseline.



DETECT AND FIX DRIFT BEFORE IT CREATES RISK.

10. PRIORITIZING FINDINGS

- Not all vulnerabilities have the same risk.
- Use severity (e.g. CVSS score), exploit availability, and business impact to prioritize.
- Focus on critical and high-risk findings first.
- Consider the asset's importance and exposure (public vs private).



FIX WHAT MATTERS MOST FIRST.

11. REMEDIATION

- Apply patches, update configurations or remove vulnerable software.
- Follow change management processes.
- Verify that the fix addresses the vulnerability.
- Document the remediation steps.
- If a fix is not immediately possible, apply compensating controls (e.g. network restrictions).



FIX. VERIFY. DOCUMENT.

12. VERIFYING THE FIX

- Rescan the system to confirm the vulnerability is resolved.
- Test that the service or application still works as expected.
- Monitor for any recurrence or new related issues.
- Keep records for compliance and audits.
- Continuous monitoring helps ensure the fix remains effective.



CONFIRM THE FIX. KEEP MONITORING.

SECURE TODAY. FEWER VULNERABILITIES. A STRONGER TOMORROW.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

CLOUD LOGGING, MONITORING, AND THREAT DETECTION



SEE MORE. DETECT FASTER. STAY SECURE.

1. WHY PREVENTION IS NOT ENOUGH

- Even with strong security controls, attacks can still happen.
- Logging and monitoring help you detect, respond and recover quickly.
- They give visibility into what is happening in your environment.
- The faster you detect a threat, the less damage it can cause.


 ASSUME BREACH.
 DETECT EARLY.
 MINIMIZE IMPACT.

2. AUDIT LOGS

- Record administrative actions (e.g. who did what and when).
- Examples: AWS CloudTrail, Azure Activity Logs, GCP Audit Logs.
- Help with security monitoring, compliance and investigations.


 KEEP AUDIT LOGS
 ENABLED AND
 PROTECTED.

3. AUTHENTICATION LOGS

- Show user sign-ins, failed logins and authentication events.
- Help detect brute force attacks, suspicious logins and unauthorized access.
- Examples: AWS CloudTrail, Azure Sign-in Logs, Okta logs, Linux auth logs.


 MONITOR
 FAILED LOGINS
 AND UNUSUAL
 SIGN-IN ACTIVITY.

4. CLOUD API ACTIVITY

- Logs all API calls made to your cloud environment.
- Helps detect unauthorized or malicious API activity.
- Examples: AWS CloudTrail, Azure Activity Logs, GCP Audit Logs.
- Look for unusual API calls, changes to security groups, creation of new resources, etc.


 MONITOR API
 ACTIVITY FOR
 SUSPICIOUS
 CHANGES.

5. NETWORK LOGS

- Show network traffic and connections.
- Help detect suspicious IP addresses, unusual traffic patterns and data exfiltration attempts.
- Examples: VPC Flow Logs, firewall logs, load balancer logs, IDS/IPS logs.


 ANALYZE NETWORK
 LOGS FOR UNUSUAL
 TRAFFIC AND
 UNKNOWN IPS.

6. APPLICATION LOGS

- Record events from your applications (e.g. errors, user activity, system behavior).
- Help detect attacks, such as SQL injection, authentication bypass or unusual activity.
- Examples: Web server logs, application logs, Kubernetes pod logs.


 CENTRALIZE
 APPLICATION LOGS
 AND MONITOR
 FOR SUSPICIOUS
 ACTIVITY.

7. SECURITY EVENTS

- Include alerts and events related to security (e.g. malware detection, policy violations, configuration changes).
- Examples: AWS GuardDuty, Azure Security Center, GCP Security Command Center.
- Help you detect and respond to threats quickly.


 USE CLOUD NATIVE
 SECURITY SERVICES
 TO DETECT THREATS
 AUTOMATICALLY.

8. METRICS

- Provide numerical data about your infrastructure and applications (e.g. CPU, memory, network, requests).
- Help you understand system health and detect anomalies.
- Examples: Amazon CloudWatch, Azure Monitor, Google Cloud Monitoring, Prometheus.


 MONITOR KEY
 METRICS AND
 LOOK FOR UNUSUAL
 SPIKES OR DROPS.

9. ALERTS

- Notify you when something unusual or suspicious happens.
- Can be based on metrics, logs or security events.
- Examples: CloudWatch Alarms, Azure Alerts, GCP Alerting, Prometheus Alertmanager.
- Send alerts to email, Slack, PagerDuty or other channels.


 SET UP MEANINGFUL
 ALERTS TO DETECT
 REAL PROBLEMS,
 NOT JUST NOISE.

10. SIEM BASICS

- SIEM (Security Information and Event Management) collects and analyzes logs from multiple sources.
- Helps detect threats, correlate events and provide a central view of security.
- Examples: Splunk, Microsoft Sentinel, ELK (Elasticsearch, Logstash, Kibana), IBM QRadar.


 USE A SIEM TO
 GET A COMPLETE
 VIEW OF YOUR
 SECURITY EVENTS.

11. SUSPICIOUS BEHAVIOR

- Look for unusual patterns such as:
 - Logins from new countries
 - Multiple failed login attempts
 - Unusual API calls
 - Large data transfers
 - Access at odd hours
 - Creation of unexpected resources


 INVESTIGATE
 WHAT LOOKS
 SUSPICIOUS.

12. DETECTING UNUSUAL LOGINS AND API ACTIVITY

- Set alerts for logins from unknown locations.
- Monitor for impossible travel (e.g. login from two countries at the same time).
- Look for unusual API calls or mass resource creation.
- Use behavior-based detection where possible.


 DETECT EARLY.
 INVESTIGATE
 QUICKLY.
 TAKE ACTION.

13. MENTAL MODEL


 VISIBILITY
 LEADS TO
 DETECTION.
 DETECTION
 LEADS TO A SAFER
 CLOUD.

LOG TODAY. DETECT TOMORROW. A SAFER CLOUD WITH VERIQTA.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

COMMON CLOUD ATTACKS AND MISCONFIGURATIONS



SMALL MISTAKES CAN LEAD TO A BIG BREACH.

1. STOLEN CLOUD CREDENTIALS

- Attackers steal access keys, passwords or tokens (e.g. from laptops, Git repos or phishing).
- They use the credentials to access your cloud account.
- Can lead to full account take over.



USE MFA. ROTATE KEYS. NEVER SHARE CREDENTIALS.

2. PUBLIC STORAGE

- Cloud storage buckets or containers accidentally made public.
- Can expose sensitive data (e.g. customer data, backups, source code).
- Often caused by incorrect bucket policies or access settings.



KEEP STORAGE PRIVATE. REVIEW PERMISSIONS REGULARLY.

3. EXPOSED DATABASES

- Databases (RDS, SQL, NoSQL) exposed to the internet.
- Weak authentication or no encryption.
- Can lead to data theft, ransomware or data destruction.



USE PRIVATE NETWORKS, STRONG AUTHENTICATION AND ENCRYPTION.

4. OPEN MANAGEMENT PORTS

- Management ports like SSH (22), RDP (3389) or Kubernetes (6443) exposed to the internet.
- Attackers can brute force or exploit vulnerabilities.
- Often caused by overly open security group rules.



RESTRICT ACCESS TO TRUSTED IPs. USE BASTION HOSTS OR VPN.

5. EXCESSIVE IAM PERMISSIONS

- Giving users, roles or services more permissions than needed.
- If compromised, attackers can access and control many resources.
- Common in both human and machine identities.



FOLLOW LEAST PRIVILEGE. REVIEW IAM ROLES REGULARLY.

6. HARDCODED SECRETS

- API keys, passwords and tokens stored directly in code.
- Can be exposed in Git repositories, logs or container images.
- Attackers can easily find and abuse them.



USE A SECRETS MANAGER. NEVER HARDCODE SECRETS.

7. UNPATCHED WORKLOADS

- Operating systems, applications or services not updated.
- Known vulnerabilities can be easily exploited.
- Common in VMs, containers and Kubernetes clusters.



KEEP SYSTEMS PATCHED. AUTOMATE UPDATES WHERE POSSIBLE.

8. INSECURE CONTAINERS

- Using untrusted base images or vulnerable dependencies.
- Running containers as root or with excessive privileges.
- Secrets stored in container images.



USE TRUSTED IMAGES. SCAN FOR VULNERABILITIES. RUN AS NON-ROOT.

9. MISCONFIGURED KUBERNETES

- Overly permissive RBAC.
- Publicly exposed Kubernetes API.
- Insecure cluster configurations.
- Privileged containers.



FOLLOW KUBERNETES SECURITY BEST PRACTICES.

10. VULNERABLE DEPENDENCIES

- Using third-party libraries with known vulnerabilities (e.g. in OS packages, language libraries or container images).
- Attackers can exploit these vulnerabilities to gain access or execute code.



SCAN DEPENDENCIES REGULARLY. KEEP THEM UP TO DATE.

11. DISABLED LOGGING

- No or limited logging and monitoring.
- Security events go undetected.
- Attackers can move around without being noticed.



ENABLE AUDIT LOGS, MONITORING AND ALERTS. SEND LOGS TO A SIEM.

12. POOR NETWORK SEGMENTATION

- Everything in the same network with no separation.
- Attackers can move laterally once inside.
- Increases the impact of a single compromised resource.



USE VPCs, SUBNETS, SECURITY GROUPS AND NETWORK POLICIES TO SEGMENT YOUR ENVIRONMENT.

13. ATTACK PATHS

- Attackers chain multiple small misconfigurations and weaknesses to achieve a big breach.
- They move from one resource to another, escalating privileges along the way.
- It is rarely just one mistake. It is the combination.



THINK LIKE AN ATTACKER. LOOK FOR HOW THINGS CONNECT.

14. EXAMPLE ATTACK PATH



Leaked Git credential (e.g. in a public repository)



Cloud account accessed (using the stolen credential)



Overprivileged IAM role (gives access to many resources)



Storage discovered (e.g. S3 bucket with sensitive data)



Sensitive data accessed and exfiltrated

15. THE BIGGER PICTURE

- All the topics from Pages 1 to 18 are connected.
- Identity, access control, networks, storage, compute, containers, Kubernetes, IaC, CI/CD, logging and monitoring all work together.
- A weakness in one area can be used to attack another area.
- A serious breach can result from several smaller security weaknesses chained together.



KEY LESSON:
A SERIOUS BREACH CAN RESULT FROM SEVERAL SMALLER SECURITY WEAKNESSES CHAINED TOGETHER.

UNDERSTAND THE RISKS. CONNECT THE DOTS. BUILD A MORE SECURE CLOUD.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



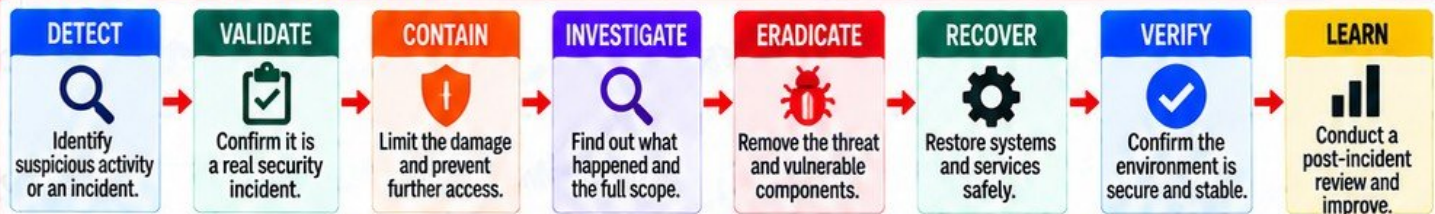
@veriqta

CLOUD SECURITY INCIDENT RESPONSE AND FINAL LAB



BE PREPARE. RESPOND EFFECTIVELY. BUILD A MORE SECURE TOMORROW.

1. INCIDENT RESPONSE LIFECYCLE



2. SUSPICIOUS LOGIN INVESTIGATION

- Check login logs (CloudTrail, Azure Sign-in Logs, etc.).
- Look for unfamiliar IP addresses, locations or devices.
- Verify if MFA was used.
- Check what actions were performed after the login.
- Identify if credentials were compromised.



ALWAYS INVESTIGATE
SUSPICIOUS ACTIVITY QUICKLY.

3. COMPROMISED CREDENTIALS

- Detect unusual logins or credential leaks (Git, CI/CD, cloud accounts).
- Assume credentials are compromised if in doubt.
- Check for unauthorized access and actions.
- Rotate and revoke immediately.
- Review where the credentials were used.



REVOKE COMPROMISED CREDENTIALS
AS SOON AS POSSIBLE.

4. ISOLATING AFFECTED RESOURCES

- Remove affected instances from load balancers.
- Disable or isolate compromised accounts.
- Apply network restrictions (security groups, NACLs).
- Quarantine containers or pods.
- Use least privilege and segmentation to limit blast radius.



CONTAIN THE THREAT.
LIMIT THE DAMAGE.

5. PRESERVING LOGS

- Do not delete or overwrite logs.
- Preserve CloudTrail, VPC Flow Logs, application logs, and system logs.
- Store logs in a secure, immutable location.
- Logs are critical for investigation and legal purposes.



PRESERVE EVIDENCE
BEFORE MAKING CHANGES.

6. DETERMINING WHAT WAS ACCESSED

- Review logs to see what resources were accessed.
- Check for data exfiltration (e.g. S3, databases).
- Identify any changes made (e.g. new users, roles, policies).
- Look for lateral movement across services.



UNDERSTAND THE FULL IMPACT
OF THE INCIDENT.

7. RESTORING SAFELY

- Remove malicious access, users or backdoors.
- Rebuild or restore from clean backups if needed.
- Patch vulnerabilities.
- Validate configurations and security settings.
- Bring services back gradually.



RESTORE A CLEAN AND
SECURE ENVIRONMENT.

8. VERIFYING RECOVERY

- Confirm the attacker no longer has access.
- Monitor for unusual activity.
- Test systems and applications.
- Validate security controls are working.
- Use vulnerability scans to confirm there are no remaining issues.



MAKE SURE THE ENVIRONMENT
IS TRULY SECURE.

9. POST-INCIDENT REVIEW

- Create a timeline of the incident.
- Identify root cause.
- Document what happened and what was learned.
- Improve policies, monitoring and controls.
- Share lessons with the team.



LEARN FROM THE INCIDENT
TO PREVENT IT HAPPENING AGAIN.

10. JUNIOR-ENGINEER SECURE ARCHITECTURE LAB



WHERE TO APPLY SECURITY CONTROLS

- | | |
|---------------|---|
| DNS | • Use secure DNS and monitoring |
| HTTPS | • Use TLS certificates (encryption) |
| WAF | • Protect against common attacks (e.g. SQLi, XSS) |
| Load Balancer | • Use security groups and network controls |
| Application | • Use IAM, least privilege, secrets management and secure configuration |
| Database | • Use encryption, private subnets, backups and access controls |

THIS ARCHITECTURE COMBINES MANY TOPICS FROM PAGES 1 - 18.
APPLY MULTIPLE SECURITY CONTROLS AT EACH LAYER.

11. FINAL MENTAL MODEL



12. KEY TAKEAWAYS

- ✓ Cloud security is a combination of people, processes and technology.
- ✓ Small misconfigurations can lead to major breaches.
- ✓ Use least privilege, encryption, monitoring and good configuration.
- ✓ Detect and respond quickly.
- ✓ Always learn from incidents and improve.
- ✓ You now have the foundation to build and manage secure cloud environments.

SECURE TODAY.
A SAFER TOMORROW
WITH VERIQTA.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta